Lab_sheet-08

Anish Tiwari

Section→C

Roll no.→11

Experiment No. 8

Experiment Name

Application of Optical Flow for Real-Time Object Tracking and Motion Analysis

Program Code and Output

Step 1: Import Required Libraries

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture(0)

if cap.isOpened():
    print("Camera opened successfully")
else:
    print("Error: Camera not opened")
```

Step 2: Shi-Tomasi Corner Detection

In [ ]:
```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, frame = cap.read()

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

corners = cv2.goodFeaturesToTrack(
    gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

if corners is not None:
    corners = np.int32(corners)

    for corner in corners:
        x, y = corner.ravel()

        cv2.circle(
            frame,
            (x, y),
            5,
            (0, 255, 0),
            -1
        )

cv2.imshow("Shi-Tomasi Features", frame)

cv2.waitKey(0)
cap.release()
cv2.destroyAllWindows()
```

Step 3: Lucas-Kanade Object Tracking

In [ ]:
```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()

old_gray = cv2.cvtColor(
    old_frame,
```

```
    cv2.COLOR_BGR2GRAY
)

# Shi-Tomasi points
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # Lucas-Kanade
    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None
    )

    if p1 is not None:

        good_new = p1[status == 1]
        good_old = p0[status == 1]

        for new, old in zip(good_new, good_old):

            x1, y1 = new.ravel()
            x2, y2 = old.ravel()

            cv2.circle(
                frame,
                (int(x1), int(y1)),
                5,
                (0, 255, 0),
                -1
            )

        p0 = good_new.reshape(-1, 1, 2)

    old_gray = frame_gray.copy()

    cv2.imshow(
        "Lucas-Kanade Tracking",
        frame
    )

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Step 4: Motion Trajectory & Displacement Vectors

In [ ]:

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

mask = np.zeros_like(old_frame)

while True:

    ret, frame = cap.read()
```

```
if not ret:
    break

gray = cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2GRAY
)

p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    gray,
    p0,
    None
)

if p1 is not None:

    good_new = p1[status == 1]
    good_old = p0[status == 1]

    for new, old in zip(good_new, good_old):

        x1, y1 = new.ravel()
        x2, y2 = old.ravel()

        x1, y1 = int(x1), int(y1)
        x2, y2 = int(x2), int(y2)

        # Trajectory
        mask = cv2.line(
            mask,
            (x1, y1),
            (x2, y2),
            (0, 255, 0),
            2
        )

        # Displacement vector
        frame = cv2.arrowedLine(
            frame,
            (x2, y2),
            (x1, y1),
            (0, 0, 255),
            2
        )

    p0 = good_new.reshape(-1, 1, 2)

output = cv2.add(frame, mask)

cv2.imshow(
    "Trajectory and Motion Vectors",
    output
)

old_gray = gray.copy()

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Step 5: Direction & Magnitude of Motion

In [ ]:

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

while True:

    ret, frame = cap.read()

    if not ret:
```

```
        break

    gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        gray,
        p0,
        None
    )

    if p1 is not None:

        good_new = p1[status == 1]
        good_old = p0[status == 1]

        for new, old in zip(good_new, good_old):

            x1, y1 = new.ravel()
            x2, y2 = old.ravel()

            dx = x1 - x2
            dy = y1 - y2

            magnitude = np.sqrt(
                dx**2 + dy**2
            )

            angle = np.degrees(
                np.arctan2(dy, dx)
            )

            cv2.arrowedLine(
                frame,
                (int(x2), int(y2)),
                (int(x1), int(y1)),
                (0, 255, 0),
                2
            )

            cv2.putText(
                frame,
                f"Speed: {magnitude:.2f}",
                (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (0, 255, 0),
                2
            )

            cv2.putText(
                frame,
                f"Direction: {angle:.1f} deg",
                (10, 60),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (0, 255, 0),
                2
            )

        p0 = good_new.reshape(-1, 1, 2)

    old_gray = gray.copy()

    cv2.imshow(
        "Motion Analysis",
        frame
    )

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Step 6: Farneback Dense Optical Flow

In [ ]:

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
```

```
old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )

    hsv = np.zeros_like(frame)

    hsv[..., 1] = 255

    hsv[..., 0] = (
        angle * 180 / np.pi / 2
    )

    hsv[..., 2] = cv2.normalize(
        magnitude,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

    result = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

    cv2.imshow(
        "Farneback Dense Optical Flow",
        result
    )

    old_gray = gray.copy()

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Step 7: Speed, Illumination, Occlusion & Camera Movement

In [ ]:

```
import cv2

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
```

```
    )

    brightness = gray.mean()

    print(
        "Average Illumination:",
        round(brightness, 2)
    )

    cv2.imshow(
        "Tracking Environment",
        frame
    )

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Step 8: Displacement, Trajectory & Average Speed

In [ ]:

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=50,
    qualityLevel=0.3,
    minDistance=7
)

total_distance = 0
frame_count = 0

while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        gray,
        p0,
        None
    )

    if p1 is not None:

        good_new = p1[status == 1]
        good_old = p0[status == 1]

        for new, old in zip(good_new, good_old):

            distance = np.linalg.norm(
                new - old
            )

            total_distance += distance

        frame_count += 1

        if frame_count > 0:
            average_speed = (
                total_distance / frame_count
            )
        else:
            average_speed = 0

        cv2.putText(
            frame,
            f"Average Speed: {average_speed:.2f}",
            (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX,
```

```
            0.7,
            (0, 255, 0),
            2
        )

        p0 = good_new.reshape(-1, 1, 2)

    old_gray = gray.copy()

    cv2.imshow(
        "Motion Parameters",
        frame
    )

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Step 9:Optical Flow vs Frame-by-Frame Detection

In [6]:
```
print("Optical Flow Tracking")
print("----------------------")
print("Tracks motion between consecutive frames.")
print("Uses feature movement.")
print("Suitable for continuous motion tracking.")

print("\nFrame-by-Frame Detection")
print("-------------------------")
print("Detects objects independently in each frame.")
print("Does not directly depend on previous frame motion.")
print("Can require more processing for repeated detection.")
```

```
Optical Flow Tracking
----------------------
Tracks motion between consecutive frames.
Uses feature movement.
Suitable for continuous motion tracking.

Frame-by-Frame Detection
-------------------------
Detects objects independently in each frame.
Does not directly depend on previous frame motion.
Can require more processing for repeated detection.
```

Step 10: Observations and Discussion

In [7]:
```
print("OBSERVATIONS")
print("-----------------------------")
print("1. Optical Flow tracks object movement.")
print("2. Shi-Tomasi detects useful feature points.")
print("3. Lucas-Kanade tracks selected feature points.")
print("4. Farneback provides dense motion information.")
print("5. Motion direction and displacement can be estimated.")
print("6. Fast motion can reduce tracking accuracy.")
print("7. Occlusion can cause feature tracking loss.")
print("8. Camera movement affects optical flow.")
print("9. Optical Flow is useful in surveillance and robotics.")
print("10. It can also support human activity recognition.")
```

```
OBSERVATIONS
-----------------------------
1. Optical Flow tracks object movement.
2. Shi-Tomasi detects useful feature points.
3. Lucas-Kanade tracks selected feature points.
4. Farneback provides dense motion information.
5. Motion direction and displacement can be estimated.
6. Fast motion can reduce tracking accuracy.
7. Occlusion can cause feature tracking loss.
8. Camera movement affects optical flow.
9. Optical Flow is useful in surveillance and robotics.
10. It can also support human activity recognition.
```

Viva / Theory Questions

Q1. How does object tracking differ from object detection?

Object Detection identifies and locates an object in an image/frame.Object Tracking follows the same object across multiple consecutive frames.

Q2.How can Optical Flow be used for real-time object tracking?

Optical Flow calculates the movement of pixels between consecutive frames. By tracking selected feature points, the position, direction and movement of an object can be continuously estimated.

Q3. Role of Shi-Tomasi Corner Detection?

Shi-Tomasi identifies strong and reliable corner/feature points in an image. These points are then used by the Lucas-Kanade algorithm to track object movement.

Q4. Why is Optical Flow suitable for motion analysis?

Optical Flow provides information about pixel movement between consecutive frames, making it useful for analyzing object direction, displacement, trajectories and motion patterns.

Q5.Challenges with fast-moving or partially occluded objects

Main challenges are:

Large displacement between frames Motion blur Loss of feature points Occlusion Reduced tracking accuracy

The experiment specifically considers object speed and occlusion as factors affecting accuracy.

Q6.Optical Flow vs Deep Learning-based Tracking

Optical Flow: Lightweight, works directly from motion between frames, and can be computationally efficient.

Deep Learning Tracking: Uses trained neural networks and can provide stronger object recognition and tracking in complex scenes, but generally requires more computational resources.

Q7. Optical Flow in traffic monitoring and autonomous driving

Optical Flow can estimate the movement, direction and speed of vehicles and other objects. This helps in traffic monitoring, motion analysis and autonomous navigation.

Q8. Effect of camera motion on Optical Flow

Camera movement creates motion across many or all pixels in the scene. Therefore, Optical Flow may interpret camera motion as object motion, making tracking less accurate.

Q9. Five real-world applications

Traffic monitoring Autonomous driving Video surveillance Human activity recognition Sports analytics

Q10. How can Optical Flow improve surveillance, robotics and activity recognition?

Optical Flow provides motion direction, displacement and trajectory information. This helps surveillance systems track objects, robots understand movement, and vision systems recognize human activities.